

TDVI: Inteligencia Artificial

Parámetros e hiper parámetros

Licenciatura en Tecnología Digital



Agenda

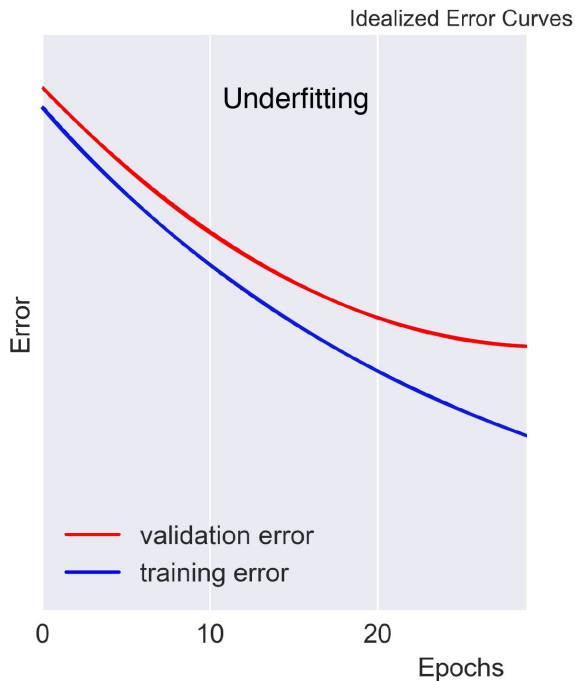
1. **Overfitting/Underfitting**
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Overfitting y Underfitting

- El objetivo más importante de machine learning es obtener buenos resultados en datos no vistos previamente (set de test).
 - La habilidad de **generalizar**
- Para esto debemos:
 - Reducir el error en entrenamiento
 - Reducir la distancia entre el error de entrenamiento y el error en test

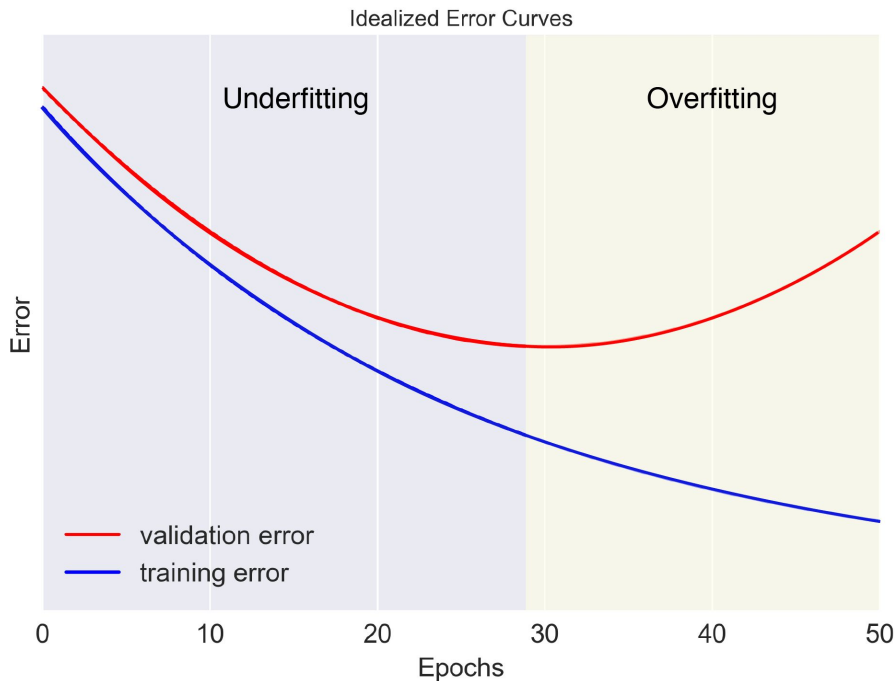
Overfitting y Underfitting

- **Underfitting:** El modelo no es capaz de bajar lo suficiente el error en entrenamiento.



Overfitting and Underfitting (cont.)

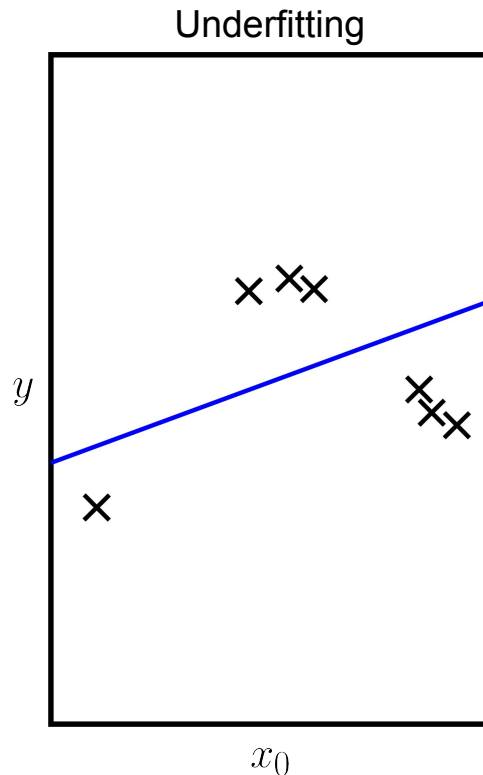
- **Underfitting:** El modelo no es capaz de bajar lo suficiente el error en entrenamiento.
- **Overfitting:** La distancia entre el error de entrenamiento y test es demasiado grande.



Capacidad del modelo

La capacidad del modelo nos permite controlar si un modelo va a tender a overfitear o underfitear

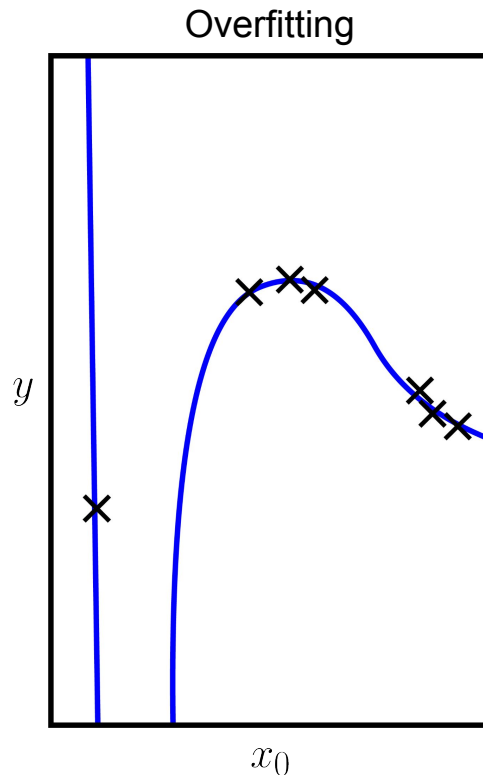
- Los modelos con baja capacidad tienen más dificultad en captar el set de entrenamiento.



Capacidad del modelo

La capacidad del modelo nos permite controlar si un modelo va a tender a overfitear o underfitear

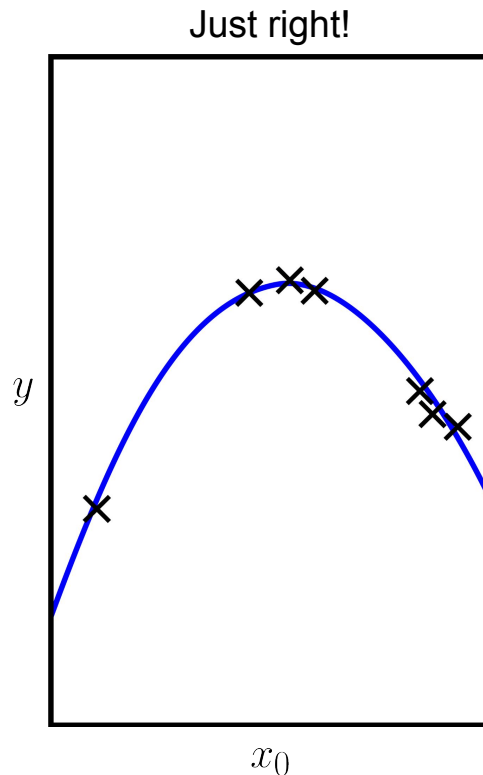
- Los modelos con baja capacidad tienen más dificultad en captar el set de entrenamiento.
- Los modelos de alta capacidad tienden a overfitear.



Capacidad del modelo

La capacidad del modelo nos permite controlar si un modelo va a tender a overfitear o underfitear

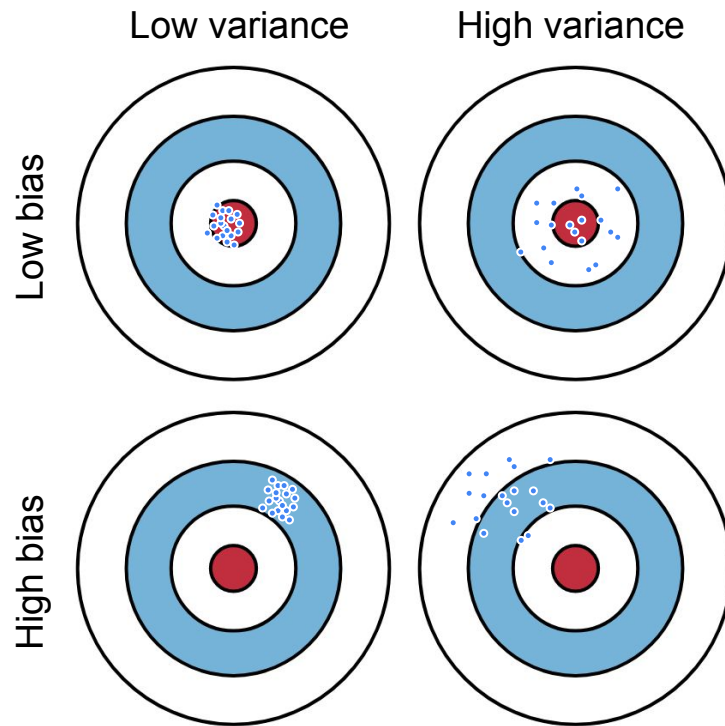
- Los modelos con baja capacidad tienen más dificultad en captar el set de entrenamiento.
- Los modelos de alta capacidad tienden a overfitear.
 - Memorizan propiedades del set de entrenamiento que le prohíben generalizar para datos nuevos



Agenda

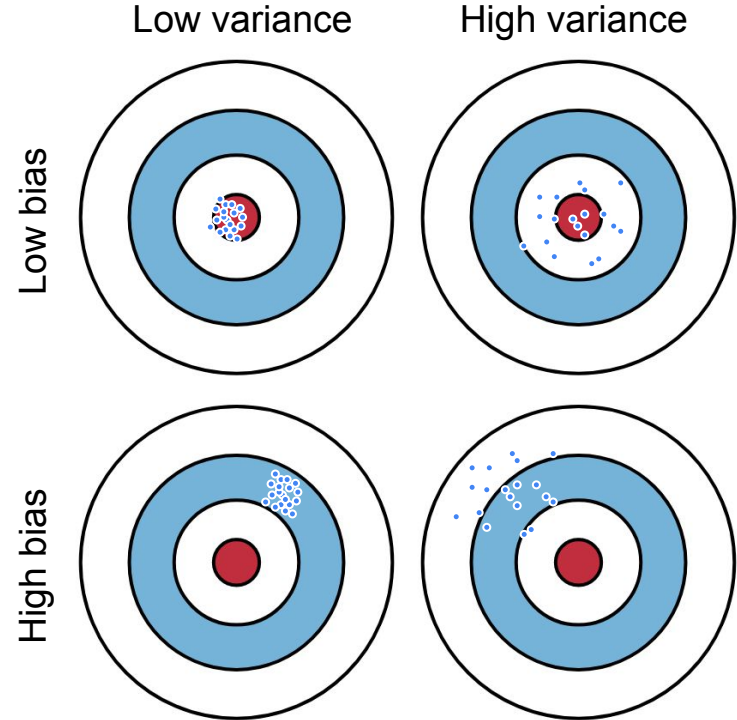
1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Bias and Variance



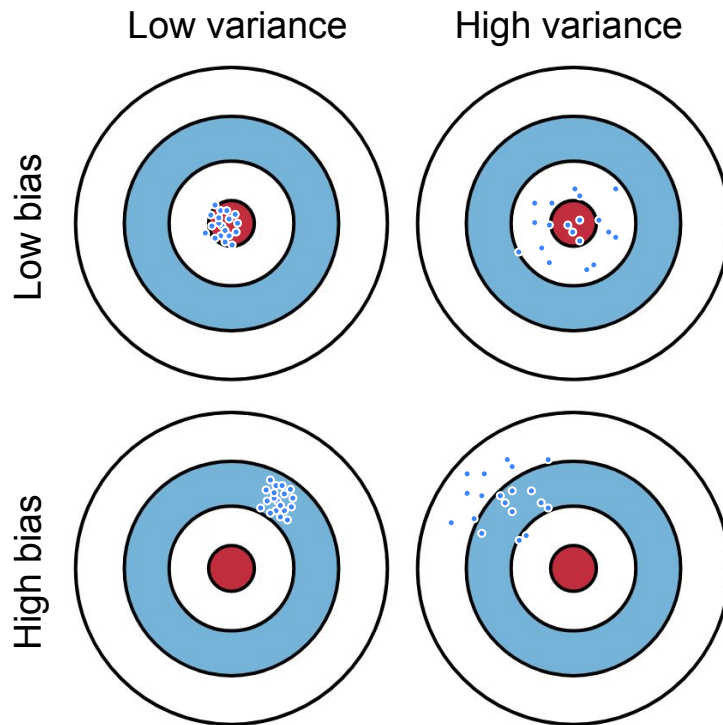
Bias and Variance

- **Bias:** la discrepancia entre las predicciones de un modelo y los valores reales observados.



Bias and Variance

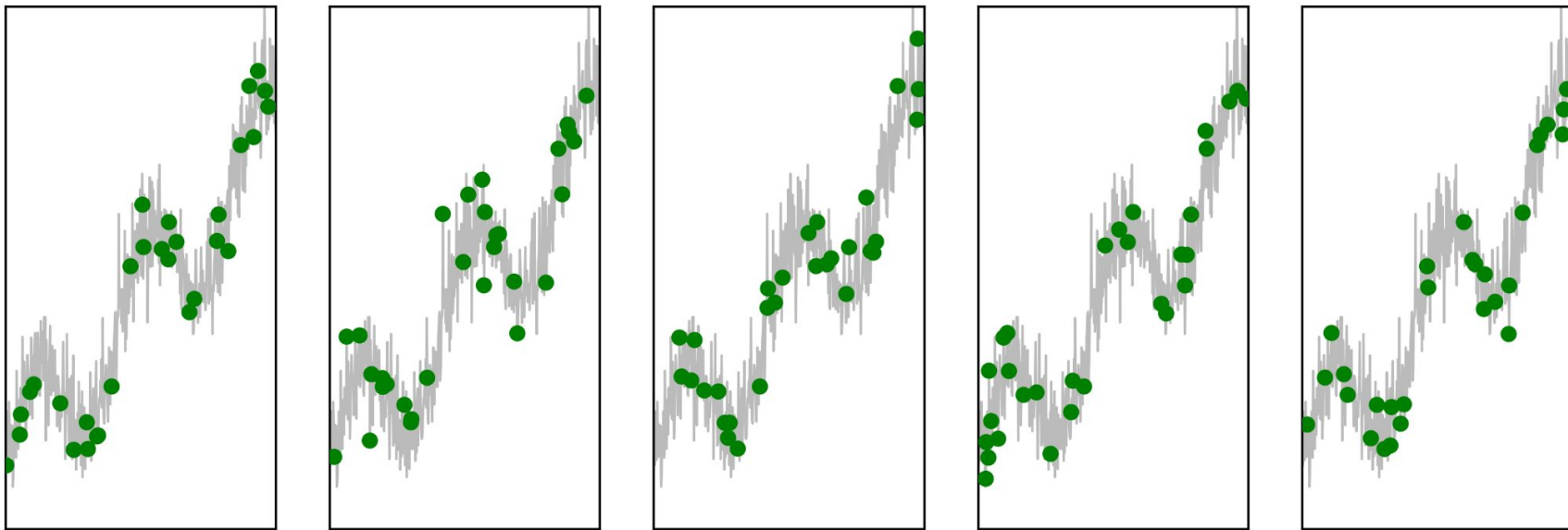
- **Bias:** la discrepancia entre las predicciones de un modelo y los valores reales observados.
- **Variance** la sensibilidad del modelo a las variaciones en los datos de entrenamiento. Puede tener un rendimiento inestable cuando se aplica a nuevos datos.



Trade-off de bias-variance

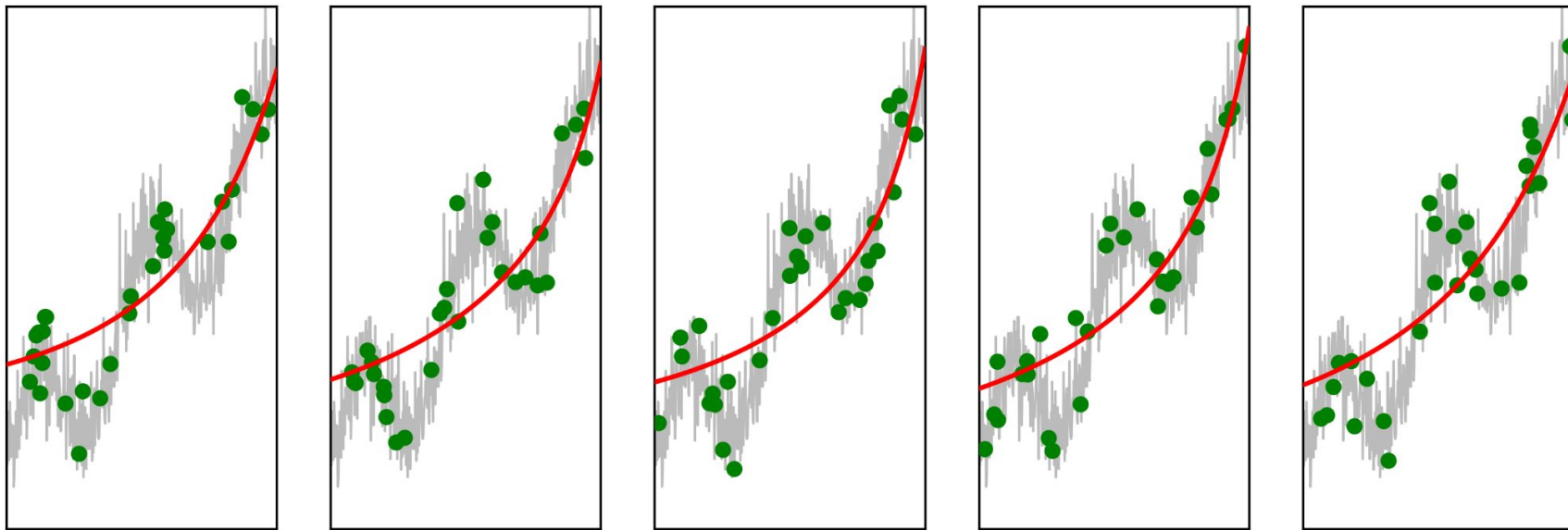
- **Demasiado bias:** El modelo es demasiado simple y no puede aprender lo suficiente de los datos.
- **Demasiada varianza:** El modelo es demasiado complejo y se ajusta demasiado a los datos de entrenamiento, fallando en generalizar.

Bias and Variance



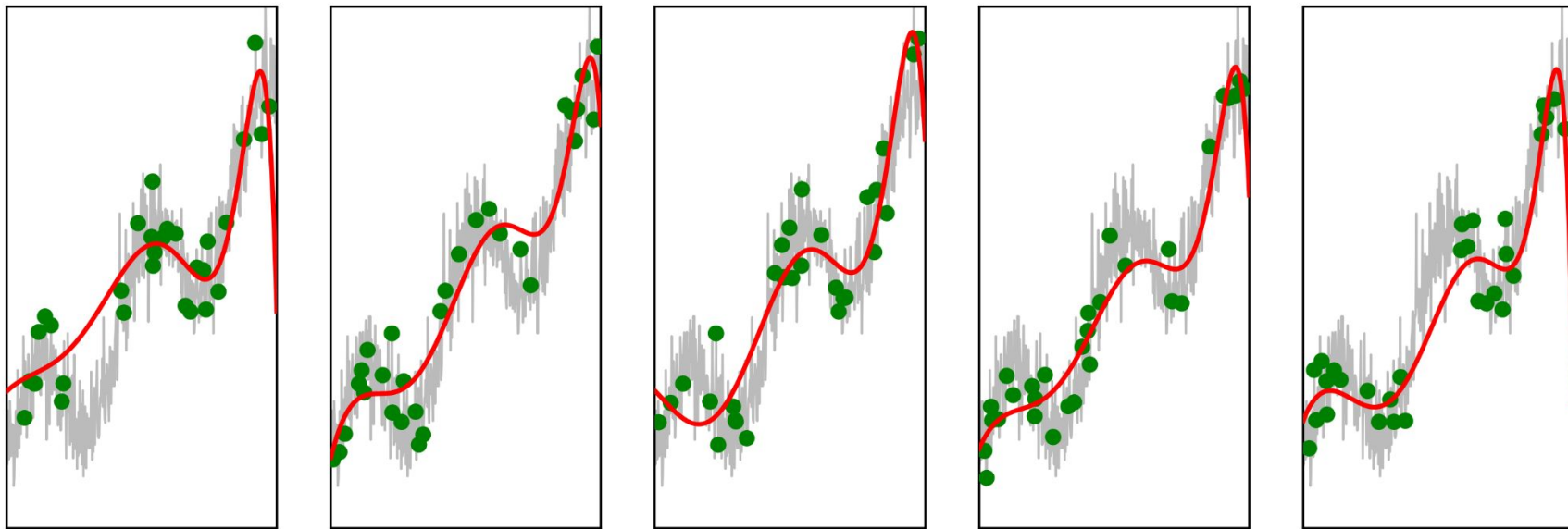
Datos de entrenamiento ruidosos

Bias and Variance



High bias, low variance

Bias and Variance



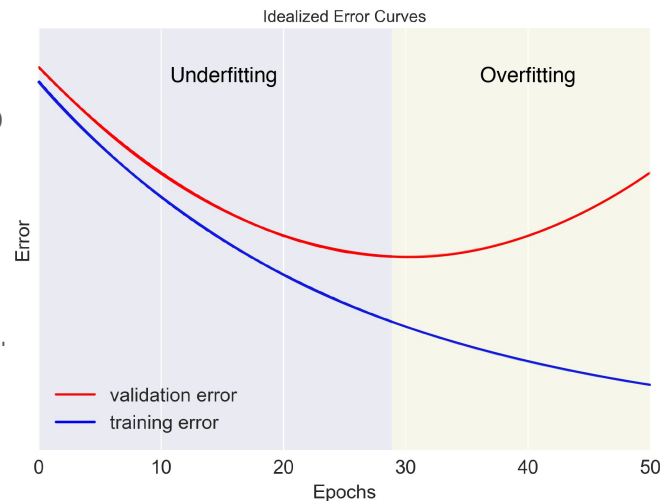
Low bias, high variance

Bias and Variance

- Bias y variance se relacionan inversamente.
- Queremos modelos con bajo bias y baja varianza
 - hay que encontrar un trade-off
- Ambos muy relacionados con la capacidad del modelo.
 - Más capacidad más varianza
 - Más capacidad menos bias

Bias and Variance

- Un modelo con **alto bias** es considerado **underfiteado**:
 - Muy simple e incapaz de capturar patrones en los datos.
- Un modelo con **alta varianza** es considerado **overfiteado**
 - Aprendió patrones de los datos de entrenamiento pero no generaliza a datos nuevos
- Un modelo con bajo bias y baja varianza está bien fiteado.
 - Es capaz de capturar patrones en los datos y generalizar a nuevos. to new, unseen inputs.



Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. **Hyperparameters**
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Hiper parámetros: ¿Qué son?

Parámetros:

- Variables internas del modelo (pesos, sesgos, etc) los cuales aprendemos durante el entrenamiento.

Hiper Parámetros:

- variables de configuración externa que determinan la estructura de la red.
- no se aprenden durante el entrenamiento
- los definimos previo al entrenamiento

Ejemplos

- Learning rate
- Cantidad de capas ocultas
- Cantidad de unidades ocultas
- Funciones de activación
- Momentum
- Tamaño del mini-batch
- Regularización

Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. **Guías**
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Disclaimer

- No hay receta mágica
- La configuración óptima va a depender de:
 - los datos
 - el problema
 - la arquitectura

Sin embargo existen algunas pautas para guiarnos

Tasa de aprendizaje (LR)

- Controla el tamaño de los updates que hacemos a los parámetros durante el descenso del gradiente.
 - LR pequeño genera convergencias más lentas, pero resulta en **mejor rendimiento**.
 - LR grande llega a convergencias más rápidas, pero **inestabilidad** y podemos pasarnos del óptimo.
- **Sugerencia:**

Considerar el uso de técnicas de **scheduling** para adaptar el LR durante las distintas iteraciones del entrenamiento.

Cantidad de capas ocultas

- Incrementar las capas:
 - Permite a la red aprender representaciones más complejas y jerárquicas de los datos.
 - Puede generar overfiteo
 - Aumenta los tiempos de entrenamiento
- **Sugerencia:**

Comenzar con números pequeños e **incrementar gradualmente**, monitoreando el rendimiento en el test de validación.

Cantidad de unidades ocultas

- Afecta la capacidad del modelo en aprender patrones complejos.

- Pocas unidades limitan la habilidad de aprender

- Demasiadas pueden causar overfitting y lentitud en el entrenamiento

$$MG = \sqrt[n]{\prod_{i=1}^n x_i}$$

- **Sugerencia:** Experimentar con distintos números de unidades ocultas

considerando la complejidad del problema y el tamaño de los datos de entrada:

- Se puede usar la media geométrica del número de neuronas de entrada y salida para encontrar un valor razonable de unidades ocultas en la primera capa oculta, equilibrando la complejidad de la red.

Funciones de activación

- Impacta el rendimiento y la convergencia del modelo
- Algunas funciones de activación tienen parámetros para tunear
- ReLU es popular:
 - Simple
 - Mitiga el vanishing gradient
- Para tareas de clasificación binaria, se puede considerar la sigmoid en la capa de salida.
- Chequear literatura para entender lo más usado para un problema similar

Momentum y tamaño del Mini-Batch

- Momentum:

- Acelera el entrenamiento
- Ayuda a salir de mínimos locales

Un valor predeterminado común que generalmente funciona bien es 0.9

- Tamaño de Mini-batch:

- Afecta el poder computacional
- Estabilidad en la convergencia

- **Sugerencia:** Experimentar con distintos valores de momentum y tamaño de mini-batch para encontrar una buena combinación para el problema específico y hardware.

Agenda

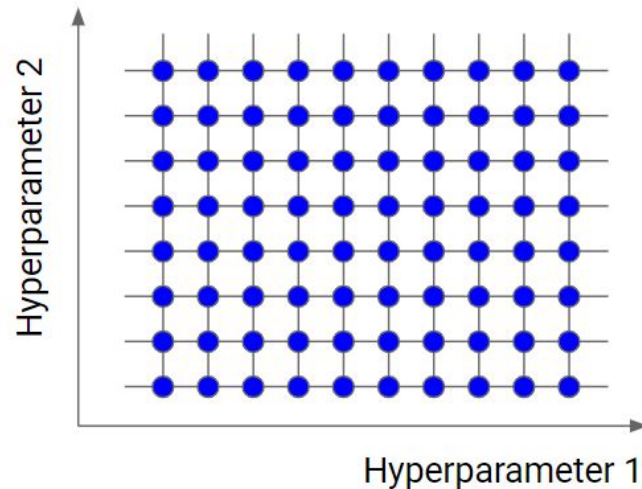
1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. **Métodos**
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Motivación

- La selección de hiper parámetros es crucial para una red de alto rendimiento.
- Distintas estrategias nos pueden ayudar a encontrar una combinación óptima de parámetros.
- Vamos a ver:
 - grid search
 - random search
 - optimización bayesiana

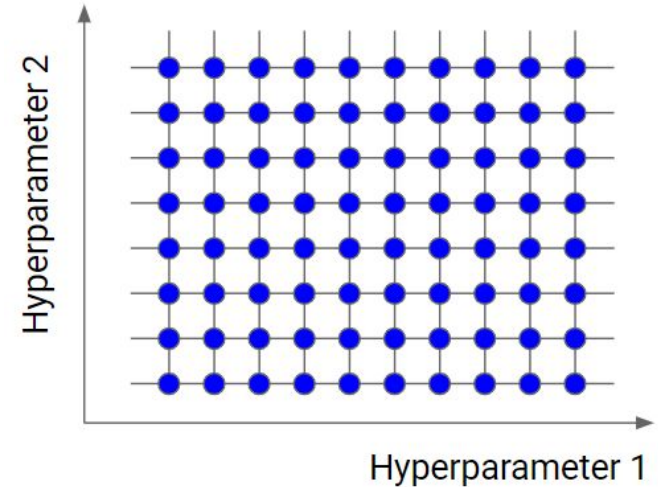
Grid Search

- Es una técnica simple y exhaustiva.
- Explora todas las posibles combinaciones de hiper parámetros en un espacio predeterminado.



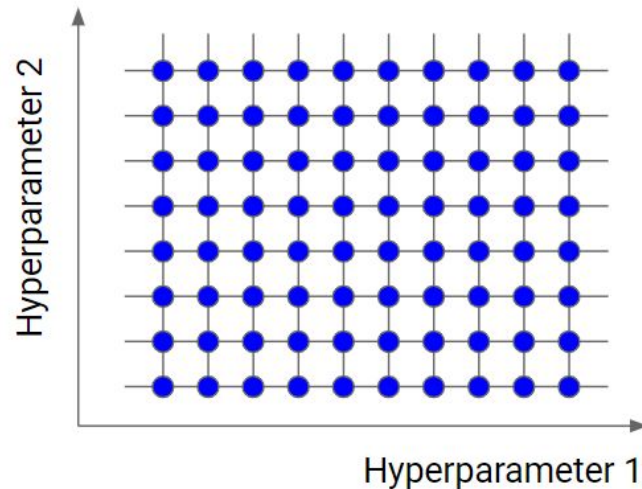
Procedimiento Grid Search

1. Definir el espacio de búsqueda para cada hiper parámetro
2. Crear grilla con las posibles combinaciones
3. Entrenar el modelo para cada combinación y evaluar en el set de validación
4. Elegir la combinación de mejor rendimiento



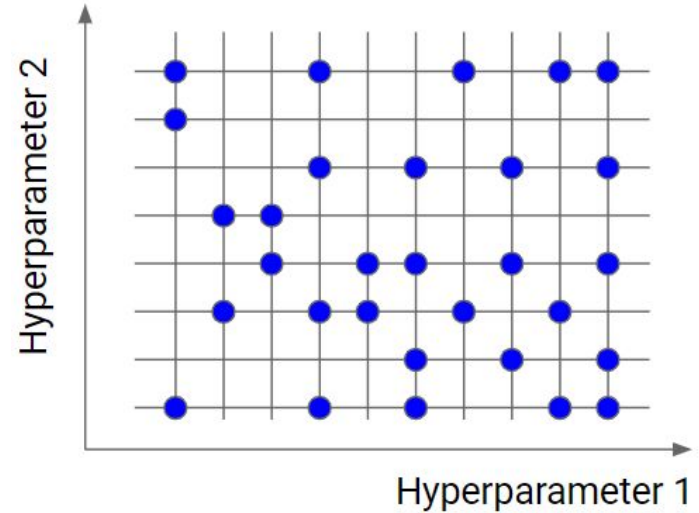
Grid Search Pros y Cons

- **Fácil de implementar**
 - Garantiza encontrar la mejor combinación en el espacio definido.
- **Computacionalmente costoso**
 - sobretodo con espacios de búsqueda amplios
- **Es eficiente para espacios continuos**



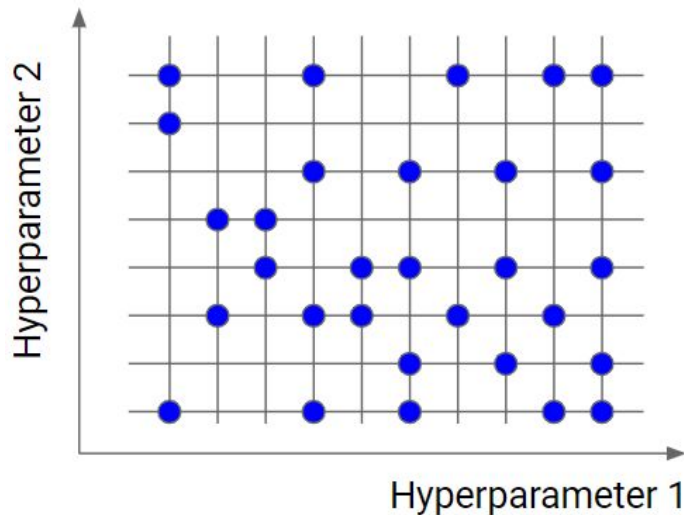
Random Search

- Explora el espacio de búsqueda de manera más eficiente
- Samplea los hiper parámetros de manera random de una distribución predeterminada



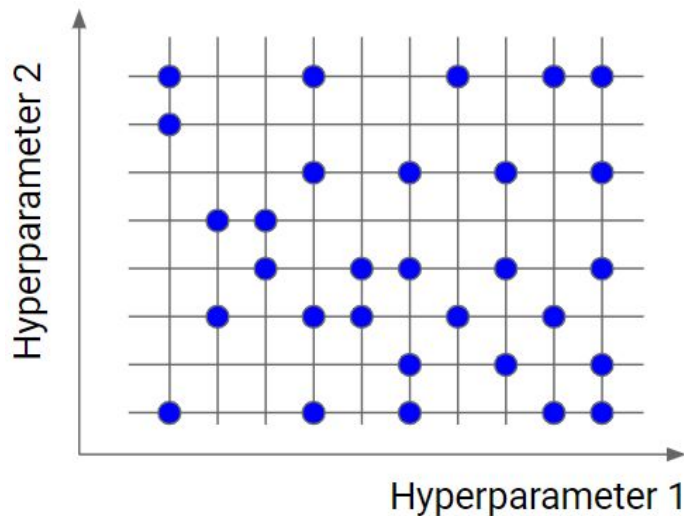
Procedimiento de Random Search

1. Definir el espacio de búsqueda y distribución de probabilidad para cada hiper parámetro
2. Samplear random valores
3. Entrenar y evaluar el modelo para cada combinación
4. Repeter el procedimiento hasta llegar a un criterio de frenada:
 - número de iteraciones
 - rendimiento



Random Search Pros y Cons

- Más eficiente que Grid Search para espacios grandes
 - Puede encontrar buenas combinaciones en menos evaluaciones
- No garantiza encontrar la mejor combinación
- Puede equivaler a grid search en el peor de los casos
- El criterio de frenada puede ser difícil de seleccionar



Optimización Bayesiana

- Más avanzada
- Técnica probabilística
- Usa prior knowledge (evaluaciones previas) para guiar la búsqueda

Optimización Bayesiana

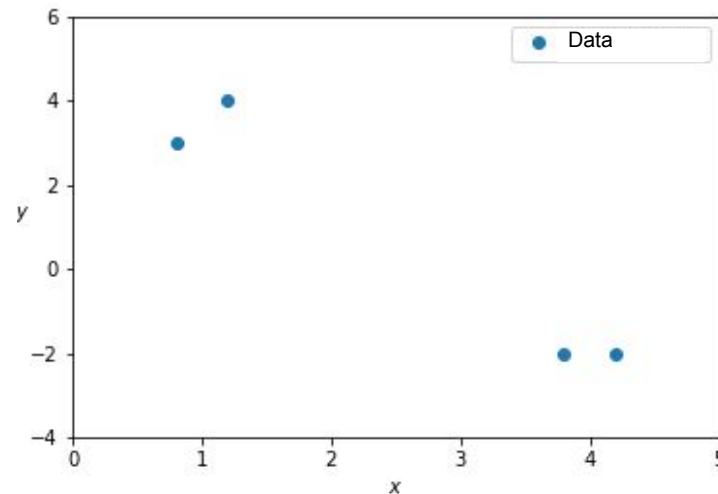
1. Definimos un espacio de búsqueda

- Ejemplo: queremos optimizar las unidades ocultas y el learning rate

Optimización Bayesiana

2. Evaluar el rendimiento del modelo en algunas combinaciones random de hiper parámetros.

Esta evaluación inicial nos da información de cuan bien para éste setting.

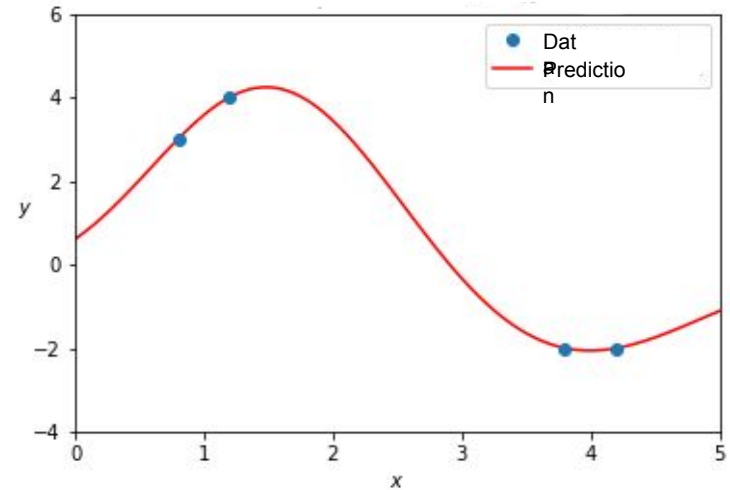


Model performance y for random values of hyperparameter x

Optimización Bayesiana

3. Ahora, en vez de seleccionar el próximo set de parámetros de manera random, usamos **un modelo probabilístico** (llamado modelo subrogado) para estimar el rendimiento de todas las posibles combinaciones de hiper parámetros basado en evaluaciones previas.

- Nos enfocamos más en la búsqueda para evitar perder tiempo en evaluaciones poco prometedoras.



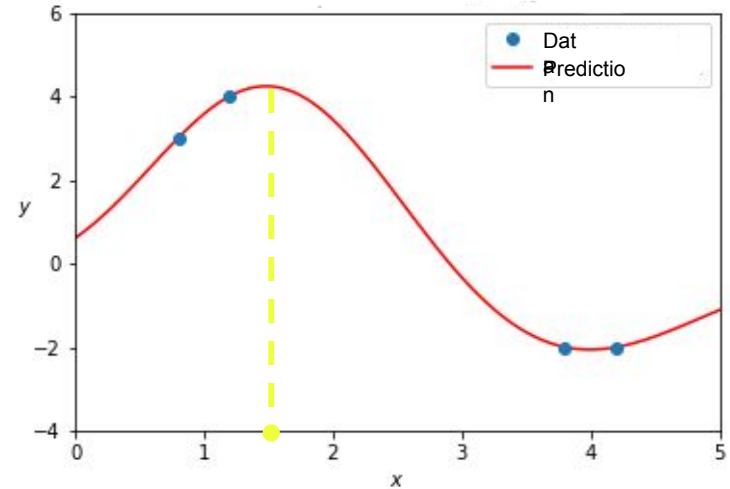
Model performance y for random values of hyperparameter x

Optimización Bayesiana

4. Seleccionar el próximo set de hiperparámetros basado en la predicción del modelo subrogado.

Funciones de adquisición:

- nuevos sets no explorados
- explotar combinaciones esperadas cerca de la solución óptima



Model performance y for random values of hyperparameter x

Optimización Bayesiana

5. Evaluar en validación y actualizar el modelo subrogado con la nueva información de performance.
6. Repetir pasos 4 y 5 hasta llegar a un criterio de frenada:
 - Máximo número de iteraciones
 - Estancamiento en la mejora del rendimiento, etc

Optimización Bayesiana Pros y Cons

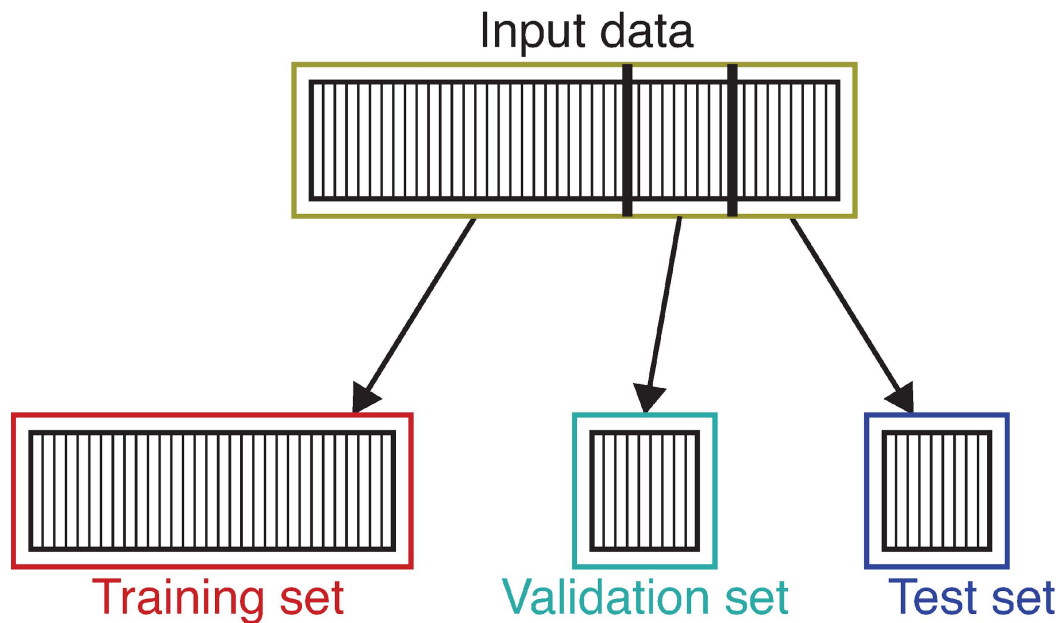
- Más eficiente que grid search y random search
- Bueno para grandes dimensiones y espacios continuous
- Se enfoca adaptativamente en regiones más prometedoras de búsqueda
- Más difícil de implementar
- Puede ser sensible a la elección del modelo subrogado y función de adquisición

Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
- 6. Training/Validation/Test Sets**
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Training/Validation/Test Sets

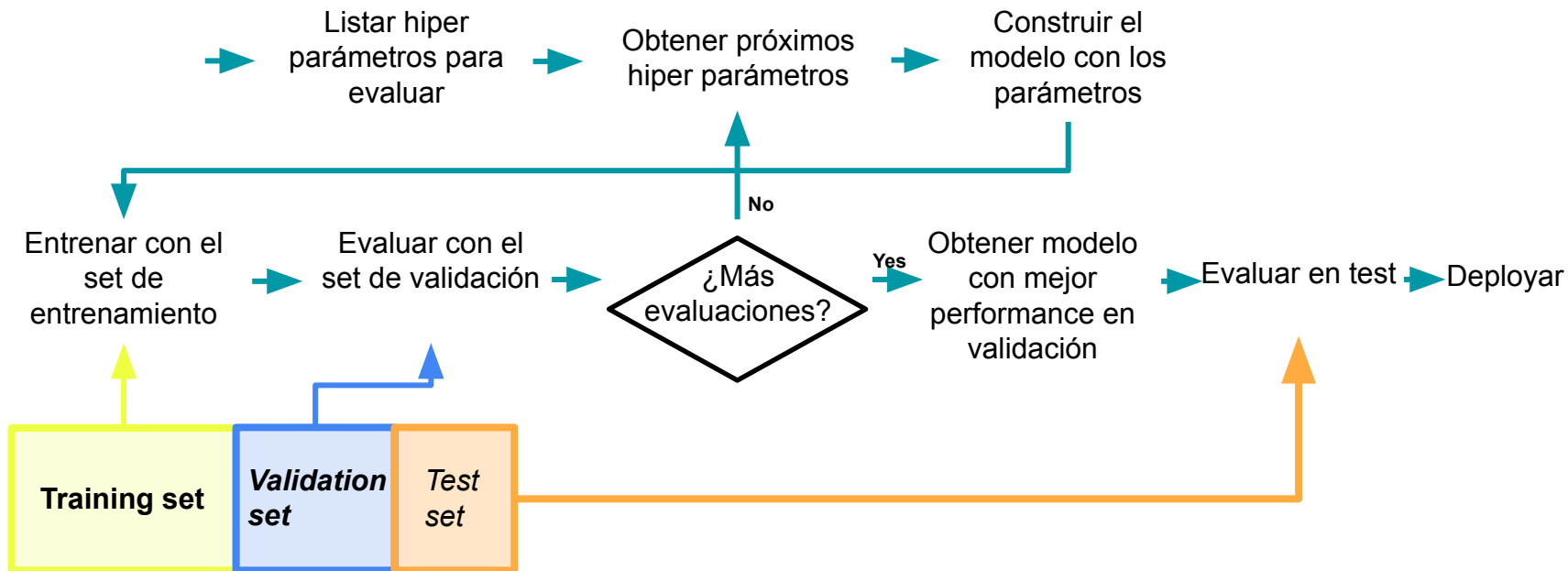
¿Cómo partimos los sets?



Crear datasets

- Asegurarse que el conjunto de entrenamiento sea representativo del mundo real
- Asegurarse de que el conjunto de validación y test tengan la misma distribución que el entrenamiento

Flujos de datos y entrenamiento



¿Qué hacer?

High bias (underfitting)

- Incrementar la NN
- Alternar arquitectura
- Entrenar más epochs

High variance (overfitting)

- Decrementar la NN
- Regularización (L1,L2, Dropout)
- Frenada temprana (menos epochs)
- Alternar arquitectura

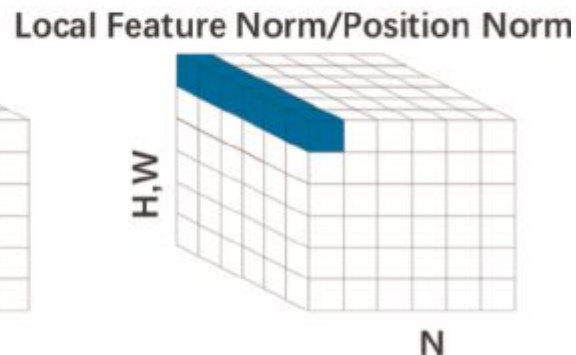
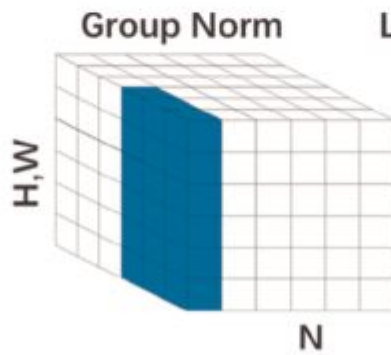
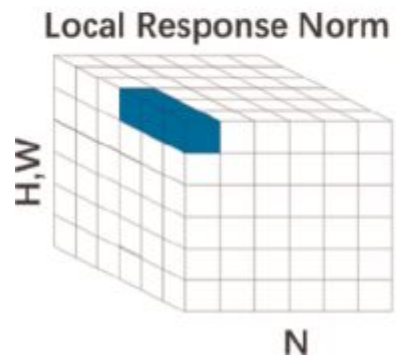
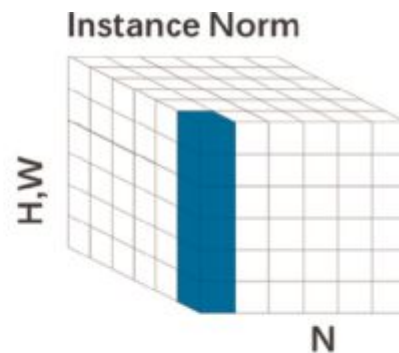
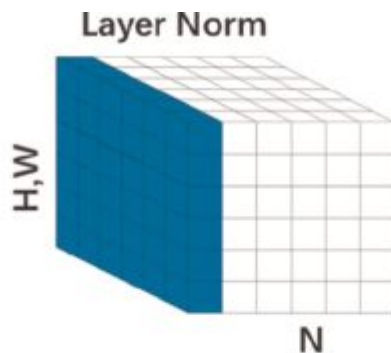
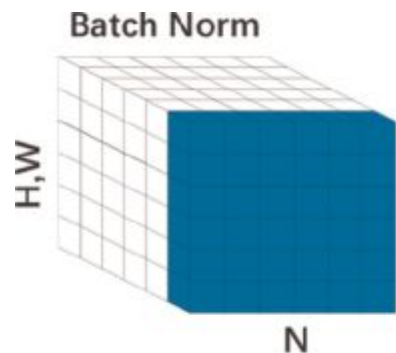
Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. **Normalizaciones**
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Input

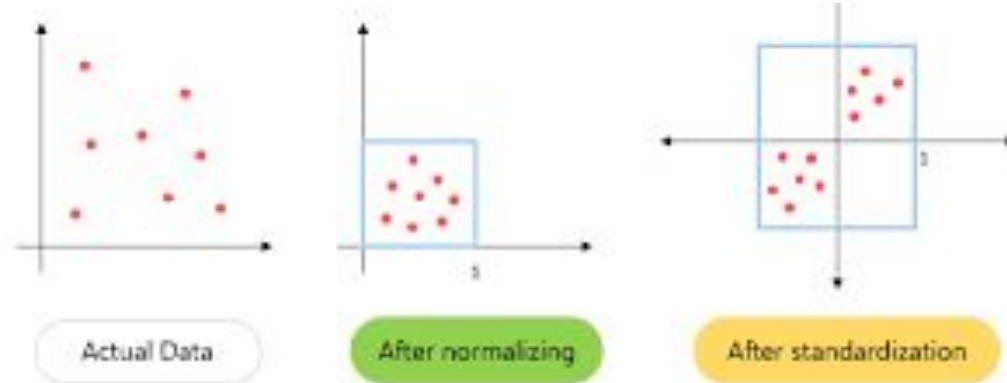
- Los features tienen distintos rangos numéricos. Ejemplo Elefante africano de sabana
 - Edad en horas (0 to 420,000)
 - Peso en toneladas (0 to 7)
 - Longitud de la cola en cm (120 to 155)
 - Edad relativa a la edad media en horas (-210,000 to 210,000)
- Normalizamos cada atributo (i.e. $[-1,1]$ or $[0,1]$).
- Estandarización (std = 1)

Normalizaciones varias...



Mini-batch

1. Calcular la media y varianza de cada feature en un mini-batch
2. Estandarizar $z = \frac{x_i - \mu}{\sigma}$



Normalización por batch: ¿por qué?

- Previene:
 - que los valores que salen de las capas sean muy positivos o muy negativos.
 - que se dispersen o comprimen demasiado
- Mantiene los valores cerca de rangos donde la función de activación funciona mejor.

Layer

- En vez de normalizar por batch, normalizamos entre features por cada sample
- Es particularmente útil para los modelos basados en secuencias, donde la entrada a menudo se procesa un elemento a la vez, y la normalización por batch es menos efectiva debido a la variabilidad en la longitud de las secuencias y el tamaño de los lotes.
 - Funciona bien con RNN y transformers

Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. **Vanishing / Exploding Gradients**
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. Bibliografía

Vanishing / Exploding Gradients

- **Vanishing Gradients:** Los gradientes se hacen muy chiquitos, alentando el aprendizaje y dificultando al modelo la actualización de pesos.
- **Exploding Gradients:** Los gradientes se hacen muy grandes, causando inestabilidad y potencialmente un entrenamiento divergente.

Inicialización de pesos

- Inicializar los pesos de la red cuidadosamente
 - No asignar los mismos pesos a todos
- Algunas técnicas de inicialización son:
 - Uniforme
 - Normal
 - LeCun Uniform/Normal
 - Xavier/Glorot Uniform/Normal

Inicialización de pesos

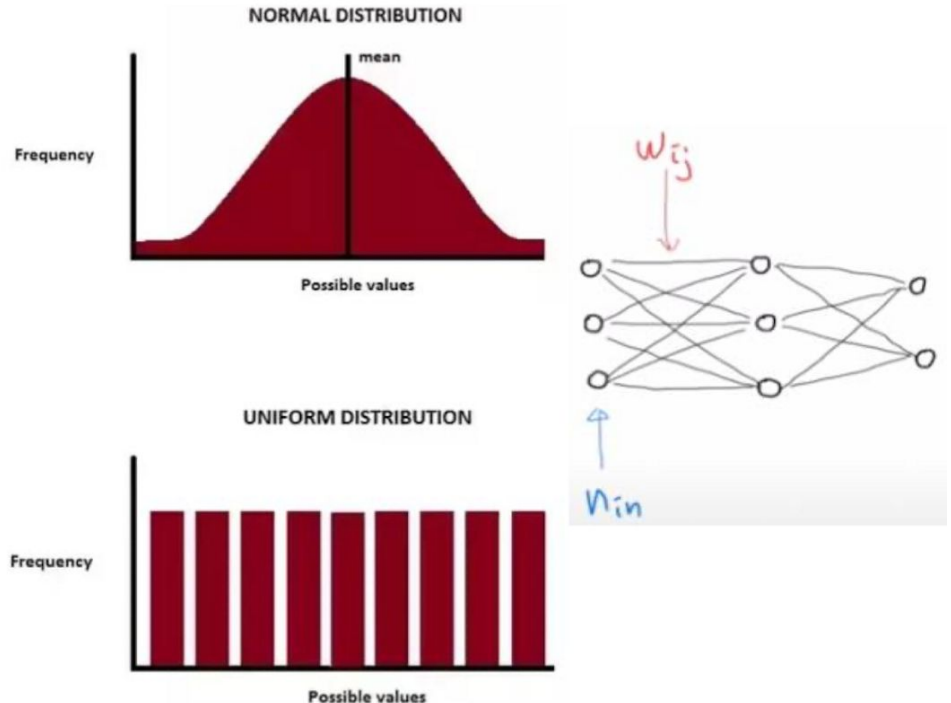
- LeCun Normal Initialization

$$W \sim N\left(0, \sqrt{\frac{1}{n_{in}}}\right)$$

- LeCun Uniform Initialization

$$W \sim U\left(-\sqrt{\frac{1}{n_{in}}}, +\sqrt{\frac{1}{n_{in}}}\right)$$

n_{in} : the number of previous node



Inicialización de pesos

Uniform Xavier initialization: draw each weight, w , from a random uniform distribution

in $[-x, x]$ for $x = \sqrt{\frac{6}{inputs + outputs}}$

Normal Xavier initialization: draw each weight, w , from a normal distribution with a mean

of 0, and a standard deviation $\sigma = \sqrt{\frac{2}{inputs + outputs}}$

Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. **Monitorear gradientes durante el entrenamiento**
10. Regularización
11. Bibliografía

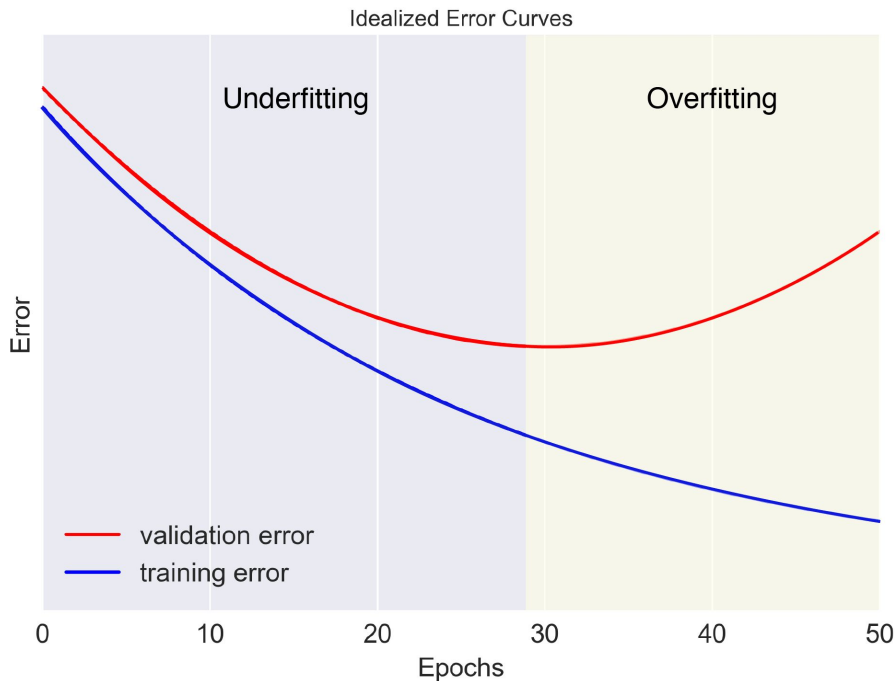
Monitorear gradientes durante el entrenamiento

- Monitorear los gradientes durante el entrenamiento
 - Identificar vanishing y exploding gradients
 - Entender el LR
 - Detectar la convergencia

Evaluar la training y validation loss

- Monitoring losses during training:

- Training loss
- Validation loss
- Compararlas es importante!!

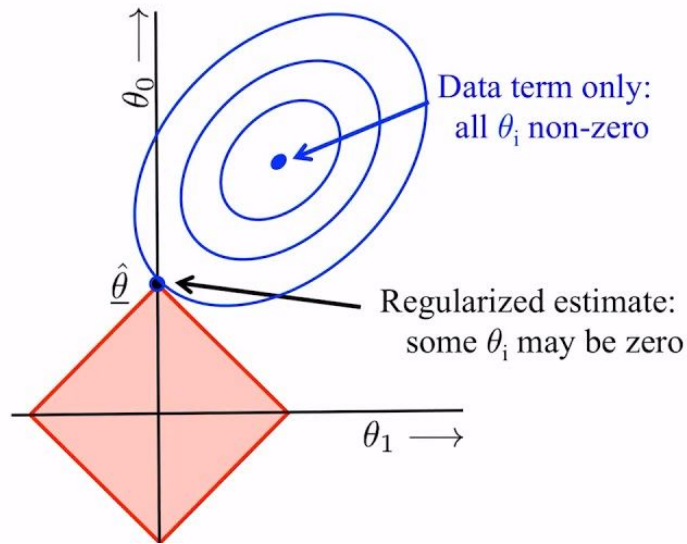


Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Métodos
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
- 10. Regularización**
11. Bibliografía

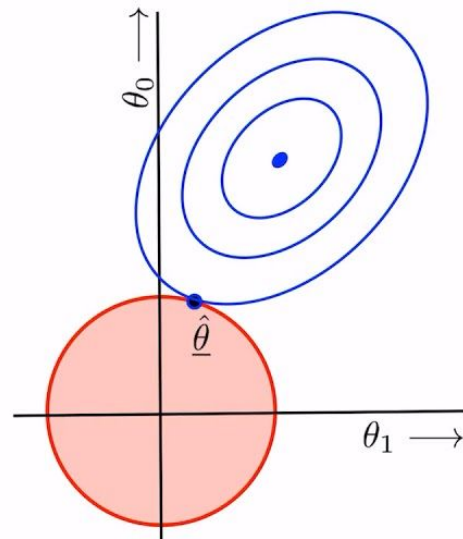
L1 Regularization

- Término L1 a la loss
- Previene overfitting agregando una penalización basada en valores absolutos
- Motiva modelos más simples y pesos más chicos
- Promueve una solución esparza
- Mejora la generalización

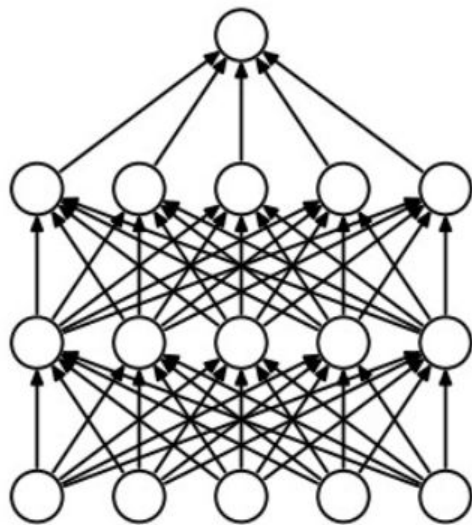


L2 Regularization

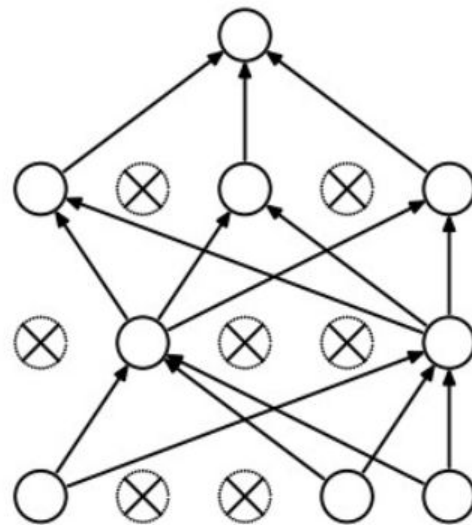
- Término L2 a la loss
- Promueve bordes más suaves
- Distribuye los pesos más parejos
- Mejora la capacidad de generalización de la red
- Mejor rendimiento



Dropouts



Without
Dropout



With
Dropout

Dropouts

- Previene overfitting
- Modifica la red en vez de la loss
- Elige random un conjunto de unidades por iteración para “desactivar”
- Mejora el rendimiento
- Típicamente elegimos valores de dropout entre .3 y .5

Otras técnicas de regularización

- Data Augmentation
 - Ejemplo imagenes: rotar, flip, aplicar transformaciones
- Noise Injection
- Early Stopping

Regularización

- L1 y L2 pueden prevenir overfitting agregando un término en la loss
- Dropout involucra desactivar unidades random lo cual promueve robustez, y generalización
- **Sugerencia:** Experimentar con distintas regularizaciones y valores de dropouts para encontrar un balance óptimo entre complejidad y generalización.

Agenda

1. Overfitting/Underfitting
2. Bias y Varianza
3. Hyperparameters
4. Guías
5. Systematic Approaches for Tuning Neural Network Hyperparameters
6. Training/Validation/Test Sets
7. Normalizaciones
8. Vanishing / Exploding Gradients
9. Monitorear gradientes durante el entrenamiento
10. Regularización
11. **Bibliografía**

Bibliografía

- [Deep Learning](#), Ian Goodfellow and Yoshua Bengio and Aaron Courville - Capítulo 4.3.1 y 8.3.1
- Neural Networks and Deep Learning. Charu C. Aggarwal 3.5